

Lab 6 - Backgrounding, Signals, Process Management

Recall how the solution to lab-5 used `Ctrl+C` to stop the loop (and the script). What is happening there? Your process was involuntarily terminated, while running. This happened because of a signal that the user sent. What if you wanted to run more commands, while this one was running? Can you change things so that the process is not interrupted by the keyboard?

One way to do this, is to put your process in the background. Open another ssh connection to veridian. Now, in the first terminal, try out the following script (call it `myscript.sh`, give it execute permissions, and run it using `./myscript.sh`).

```
#!/bin/bash

# Infinite loop
while true; do
    # Print the current date and time
    date

    # Sleep for 3 seconds
    sleep 3
done
```

All it does, is print the time and goes to sleep for `3` seconds. After `3` seconds, it wakes up and repeats the loop. In the second terminal, try out the command

`ps -u <your user id>`. You should see several processes listed. One of them should be `myscript.sh`.

Now, go back to the first terminal and stop the script using `Ctrl+C`. Now, run it again, but this time use the following command: `./myscript.sh &` Note the `&` character at the end

What happens? The script is still running (check using `ps -u <your user id>`, in the second terminal), but `Ctrl+C` has no effect. This is because the script is now running in the background, and the signals sent via the keyboard do not affect it. You can bring it back to the foreground, by typing 'fg' in the same terminal as where you ran it. Now, you can stop your script by typing `Ctrl+C`

Backgrounding is used by several processes, to run for a long time (or even, indefinitely). Read the manpage of the `ps` command to see multiple options on how to list processes.

Trapping Signals with Bash

How can we trap signals (like `Ctrl+C`) in Bash scripts? Signals are notifications sent to a process to notify it of events like interruptions or termination requests. If you recall from your OS theory lectures, a process can be in one of many states. For instance, a process can be suspended. Or it may suffer involuntary termination. Some of these states occur, when a particular event happens. The OS will send a notification to the process, allowing the process the opportunity to react gracefully to the event. In our case, trapping signals allows you to handle these events gracefully in your scripts.

What are Signals?

Signals are a form of inter-process communication in Unix-like operating systems. Some common signals include:

1. **SIGINT (2):**

- Sent when you press `Ctrl+C`. It interrupts a process, allowing it to perform cleanup operations before terminating.

2. **SIGTERM (15):**

- A termination request sent to a process. It allows the process to terminate gracefully, performing any necessary cleanup.

3. **SIGHUP (1):**

- Sent when the terminal is closed or when a controlling process dies. It is often used to indicate that a configuration file should be reloaded.

4. **SIGQUIT (3):**

- Sent when you press `Ctrl+\`. It causes the process to terminate and produce a core dump, which can be useful for debugging.

5. **SIGKILL (9):**

- Forces a process to terminate immediately. It cannot be caught or ignored, and the process does not have a chance to perform cleanup.

6. SIGSTOP (19):

- Stops (pauses) a process. Like `SIGKILL`, it cannot be caught or ignored. The process can be resumed later with `SIGCONT`.

7. SIGCONT (18):

- Resumes a process that has been stopped by `SIGSTOP` or another stop signal.

8. SIGUSR1 (10) and SIGUSR2 (12):

- User-defined signals that can be used for custom purposes in applications. They can be caught and handled by the process.

9. SIGALRM (14):

- Sent when a timer set by the `alarm()` function expires. It can be used for implementing timeouts.

10. SIGPIPE (13):

- Sent to a process when it attempts to write to a pipe that has no readers. By default, this signal causes the process to terminate.

These signals can be sent using the `kill` command followed by the process ID (PID) and the signal name or number. For example:

```
kill -SIGTERM <PID>
```

or

```
kill -15 <PID>
```

Understanding these signals is essential for managing processes effectively in a Unix-like environment.

Trapping Signals

You can use the `trap` command in Bash to specify commands that should be executed when a particular signal is received.

Syntax

```
trap 'commands' SIGNAL
```

- `commands` : The commands to execute when the signal is received.
- `SIGNAL` : The signal you want to trap (e.g., `SIGINT` , `SIGTERM`).

Example: Trapping SIGINT

Let's create a simple script that demonstrates how to trap the `SIGINT` signal.

1. Create a new Bash script:

Open your terminal and create a new file called `trap_example.sh`:

```
touch trap_example.sh
chmod +x trap_example.sh
```

2. Edit the script:

Open the file in your text editor and add the following code:

```
#!/bin/bash

# Function to handle SIGINT
cleanup() {
    echo "Caught SIGINT! Cleaning up..."
    exit 0
}

# Trap SIGINT
trap cleanup SIGINT

# Main loop
echo "Running... (Press Ctrl+C to stop)"
while true; do
    sleep 1
done
```

3. Run the script:

In your terminal, run the script:

```
./trap_example.sh
```

You should see the message "Running... (Press Ctrl+C to stop)".

4. Test the signal trapping:

While the script is running, press `Ctrl+C`. You should see the message "Caught SIGINT! Cleaning up..." before the script exits.

Example: Trapping Multiple Signals

You can trap multiple signals by specifying them in the `trap` command.

1. Modify the script:

Update your `trap_example.sh` script to trap both `SIGINT` and `SIGTERM`:

```
#!/bin/bash

# Function to handle signals
cleanup() {
    echo "Caught signal! Cleaning up..."
    exit 0
}

# Trap SIGINT and SIGTERM
trap cleanup SIGINT SIGTERM

# Main loop
echo "Running... (Press Ctrl+C or send SIGTERM to stop)"
while true; do
    sleep 1
done
```

2. Run the script:

Run the script again:

```
./trap_example.sh
```

3. Test the signal trapping:

- Press `Ctrl+C` to see the cleanup message.
- Open another terminal and find the process ID (PID) of your script using `ps`:

```
ps aux | grep trap_example.sh
```

- Use the `kill` command to send a `SIGTERM` signal:

```
kill -15 <PID>
```

You should see the same cleanup message when the script receives the `SIGTERM` signal.

Conclusion

You can extend this concept to handle other signals and perform different actions based on your needs.

Feel free to experiment with trapping other signals and adding more complex cleanup logic to your scripts!